Entities

1. **Products** - Each product belongs to a single category.
2. **Providers** - Companies from which we buy products. Each provider is associated with specific categories of products.
3. **Batches** - Groups of products purchased from providers.
4. **Storages** - Locations where purchased products are stored for future sales to clients.
5. **Clients** - Companies (e.g., markets, supermarkets, shops) to which we sell products for profit.

How It Works

1. Products are purchased in **batches** from **providers** and stored in **storages**.
2. Products are sold to **clients** at increased prices for profit.
3. Refunds can be made at two levels:
    - Refunds to providers for purchased products (fully or partially).
    - Refunds from clients for sold products (fully or partially).

Task Requirements

1. Design a **database architecture** for the described workflow.
    - **Categories**:
        - Support parent-child relationships.
        - Root category is connected to a specific provider.
          Example:
          Ahmad Tea (Root Category) → Black Tea (Child Category), Green Tea (Child Category), White Tea (Child Category).
        - Products belong to a specific category.
          Example:
          "Ahmad Tea Earl Grey, 500g" → Black Tea Category.
    - **Batches**:
        - Each batch can contain multiple products with varying quantities.
        - Refunds should be supported for batches (fully or partially).
    - **Storages**:
        - Products are stored here after purchase and sold from these storages.
    - **Clients**:
        - Products are sold to clients.
        - Refunds for sold products (fully or partially) should be supported.
2. Implement the following **API endpoints**:
    - **[POST] Purchase Products and Add to Storage**

        - Enables buying products from providers and storing them in the appropriate storage.
    - **[POST] Refund Purchased Products on Batches**
        - Allows refunds for unsold products in batches.
        - Refunded products are deducted from the storage.
    - **[GET] Fetch Available Products for Ordering**
        - Returns a list of available products for ordering.
        - Response should include:
            - `id` (Product ID)
            - `name` (Product Name)
            - `category_name` (Category Name)
            - `price` (Product Price)
            - `qty` (Available Quantity).
    - **[POST] Create Client Orders**
        - Allows creating orders for clients.
        - Automatically assigns the appropriate batch for each product based on the oldest available stock.
        - The frontend does not need to send `batch_id`. The backend handles this mapping.
        - **Request Parameters**:
            - `client_id` (integer): ID of the client placing the order.
            - `products` (array): List of products in the order, where each product includes:
                - `id` (integer): Product ID.
                - `qty` (integer): Quantity ordered.
    - **[GET] Calculate Remaining Quantities in Storage**
        - Calculates and returns remaining quantities of products in storage within a specified date.
        - **Request Parameters**:
            - `date`
    - **[GET] Calculate Profit per Batch**
        - Calculates profit for each batch, factoring in refunds for accurate results.

Technical Requirements

- Use **Laravel** for the backend.
- Use **MySQL** for the database.